Korea 2026 Trip Companion — Technical Overview

A complete AI-powered trip companion system for a 12-day family trip to South Korea, consisting of a web application, Telegram bot, and proximity alert system (WanderGuide).

System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    USER DEVICES                          │
│                                                          │
│  📱 iPhone                    💻 Desktop                 │
│  ├── Safari (Web App)         └── Chrome (Web App)       │
│  ├── Telegram (GPS + Admin)                              │
│  ├── Pushover (Push Alerts)                              │
│  └── AirPods (Siri Readout)                              │
└──────────────┬──────────────────────────┬────────────────┘
               │                          │
               ▼                          ▼
┌──────────────────────┐  ┌──────────────────────────┐
│  GitHub Pages        │  │  Railway.app             │
│  (Static Hosting)    │  │  (Node.js Server)        │
│                      │  │                          │
│  • index.html        │  │  • Express API Server    │
│  • Voucher PDFs      │  │  • Telegram Bot (Telegraf)  │
│  • Free, auto-deploy │  │  • WanderGuide Proximity │
│                      │  │  • Scheduled Messages    │
│  URL: eyaniv1.github.io │  │                       │
│  /korea-2026-bot/    │  │  URL: korea-2026-bot-    │
│                      │  │  production.up.railway.app │
└──────────────────────┘  └──────────────┬───────────┘
                                          │
                     ┌────────────────────┼────────────────┐
                     ▼                    ▼                ▼
              ┌──────────────┐  ┌──────────┐  ┌──────────────┐
              │ Anthropic    │  │ Groq     │  │ Pushover     │
              │ Claude API   │  │ Whisper  │  │ Notifications │
              │              │  │          │  │              │
              │ AI Chat,     │  │ Voice    │  │ Native push  │
              │ Web Search,  │  │ message  │  │ to iOS/Watch │
              │ Photo Vision │  │ to text  │  │ + Siri readout│
              └──────────────┘  └──────────┘  └──────────────┘
```

Hosting & Services

Service	Purpose	URL/Location	Cost	Account
GitHub Pages	Hosts the web app (HTML + voucher PDFs)	eyaniv1.github.io/korea-2026-bot/	Free	eyaniv1 on GitHub
Railway.app	Hosts Node.js server (bot + API)	korea-2026-bot-production.up.railway.app	~$5/month (Hobby plan)	Linked to GitHub
Anthropic Claude API	AI brain for chat, photo analysis, web search	console.anthropic.com	Pay-per-use (~$0.01-0.05/message)	API key in Railway env vars

Groq	Voice message transcription (Whisper)	console.groq.com	Free tier	API key in Railway env vars
Pushover	Native push notifications to iOS	pushover.net	$5 one-time per device	App token + per-user keys
Telegram	Bot platform + background GPS tracking	telegram.org	Free	Bot token from @BotFather

---

API Keys & Environment Variables

Stored as environment variables on Railway (never in code):

Variable	Service	How to obtain
`TELEGRAM_BOT_TOKEN`	Telegram	@BotFather → /newbot
`ANTHROPIC_API_KEY`	Claude AI	console.anthropic.com → API Keys
`GROQ_API_KEY`	Groq Whisper	console.groq.com → API Keys

Pushover keys are stored in bot.js code (app token) and in server memory (user keys registered via /register command).

GitHub Repository

Field	Value
URL	github.com/eyaniv1/korea-2026-bot
Visibility	Public (required for GitHub Pages)
Branch	master
Auto-deploy	Railway watches master branch — every push triggers redeploy
GitHub Pages	Enabled, serves from root of master

Server (Railway) — What It Runs

A single Node.js process (`bot.js`) that runs both:

1. Telegram Bot (Telegraf)

- Long polling connection to Telegram API
- Handles: text messages, voice messages, photos, location shares, commands
- Per-user location tracking for WanderGuide
- Scheduled messages via node-cron (morning briefing, evening reminder, photo prompt)

2. Express HTTP API

Endpoint	Method	Purpose
`/api/chat`	POST	Web app chat — sends message to Claude, per-user sessions via X-Session-Id header. Handles text + image (base64).
`/api/pois`	GET	Returns full POI database (static 80+ places + custom POIs)

`/api/pois`	POST	Add a custom POI (name, lat, lng, desc)
`/api/pois`	DELETE	Clear all custom POIs
`/api/alerts`	GET	Returns proximity alerts since a given timestamp (for web app polling)
`/api/broadcasts`	GET	Returns scheduled broadcast messages since a given timestamp
`/api/pushover/register`	POST	Register a Pushover user key (from web app)
`/api/pushover/users`	GET	List registered Pushover users
`/api/health`	GET	Health check — POI count, session count

Dependencies (npm)

```
@anthropic-ai/sdk    — Claude API client
telegraf              — Telegram bot framework
express               — HTTP API server
cors                  — Cross-origin requests
groq-sdk              — Groq Whisper API client
node-cron             — Scheduled message timing
dotenv                — Environment variable loading
```

WanderGuide — Proximity Alert System

How It Works

1. User shares **live location** in Telegram DM with the bot
2. Telegram sends GPS updates every 30-60 seconds to the server
3. Server compares each user's position against the POI database using **Haversine formula**
4. When a user is within the alert radius (default 300m) of a POI:
    - **Pushover notification** sent to that specific user (native iOS push)
    - **Telegram DM** sent to that user
    - **Alert queued** for web app display
5. Web app polls the alert queue every 15 seconds and shows new alerts in the chat

Anti-Spam Controls

Setting	Default	Configurable via
Alert radius	300m	Telegram /radius
Cooldown between any alerts	20 min	Telegram /cooldown
Re-alert same POI after	4 hours	Telegram /realert

POI Database

- **Static:** 80+ places in poi-database.js (Seoul, Jeju, Busan, Gyeongju)
- **Custom:** Added via Telegram /addpoi or web chat natural language
- Categories: temple, palace, market, food, nature, culture, shopping, nightlife, viewpoint, hidden-gem
- Each POI: name, lat, lng, description, city, category

Web Application (index.html)

A single HTML file (~1200 lines) serving as a mobile-first Progressive Web App.

Pages (show/hide, not SPA routing)

Page	Content
Home	Cover page — flag, family name, dates, Korean proverb
23/4 through 4/5	12 day pages, each with 3 tabs (Itinerary, Hotel, Transport)
To-Do	Persistent checkbox list of all reservations and pre-trip tasks
Info	WanderGuide setup instructions, payment guide, transit card info
Missing?	Places not in the itinerary but worth considering

Key Features

- **Interactive maps** (Leaflet.js + CARTO tiles) with numbered POI markers
- **POI info modals** with description, logistics, booking requirements
- **Voucher PDF links** on relevant days (hotel, flight, car, attraction confirmations)
- **Rainy day alternatives** for every day
- **Floating AI chat** (💬 button) with Claude-powered assistant
- **Photo analysis** — snap a photo of a Korean sign/menu → Claude translates
- **Persistent checkboxes** — ToDo items saved in localStorage
- **Per-user chat sessions** — each person gets their own conversation
- **Chat message history** — last 100 messages persisted in localStorage
- **Proximity alert display** — server alerts appear in chat with sound

Client-Side Storage (localStorage)

Key	Content
`chat_session`	Random session ID for per-user chat
`chat_messages`	Last 100 chat messages (visible history)
`todo_<id>`	Individual ToDo checkbox states

External Dependencies (CDN)

- Leaflet.js 1.9.4 (maps)
- CARTO Voyager tiles (map imagery)

Data Flow Diagrams

Chat Message Flow

```
User types in web chat
    → POST /api/chat (with X-Session-Id header)
    → Server adds to per-session history
    → Server calls Anthropic Claude API (with trip context + web search)
    → Claude responds
    → Server returns reply
    → Web app displays in chat bubble
    → Message saved to localStorage
```

Proximity Alert Flow

```
Telegram live location update
    → Server receives GPS for user
    → Server runs Haversine distance check against POI database
    → POI within radius + cooldown passed:
        ├── Pushover API → native notification on user's phone
        ├── Telegram API → DM message to user
        └── Alert queue → web app polls and displays in chat
```

Scheduled Message Flow

```
node-cron fires at 8am/8pm/10pm KST
    → Server generates prompt for Claude
    → Claude creates morning briefing / evening reminder / photo prompt
    → Message sent to:
        ├── All active Telegram groups
        └── Broadcast queue (web app polls every 60 seconds)
```

Voucher Access Flow

```
User asks "show me the car rental voucher" in web chat
    → Server sends to Claude (system prompt includes all voucher paths)
    → Claude responds with markdown link to the PDF
    → Web app renders clickable link
    → User taps → PDF opens in new tab (hosted on GitHub Pages)
```

Security Considerations

Item	Status
API keys	Stored as Railway environment variables, not in code
Pushover app token	In bot.js source code (public repo) — acceptable for personal use
Pushover user keys	Stored in server memory, not persisted
GitHub repo	Public (required for GitHub Pages free hosting)
Voucher PDFs	Public (accessible via GitHub Pages URL) — contains booking confirmations
CORS	Enabled on all API endpoints (any origin)
Authentication	None — API endpoints are open. Acceptable for personal trip use.

Note: This system is designed for personal/family use, not for production SaaS. The open API endpoints and public vouchers are acceptable tradeoffs for simplicity. For a multi-tenant production system, authentication, encryption, and private hosting would be required.

Deployment & Maintenance

Deploying Changes

1. Edit files locally (via Claude Code or any editor)
2. `git add . && git commit -m "description" && git push`
3. GitHub Pages updates automatically (1-2 min)
4. Railway redeploys automatically (1-2 min)

What Survives a Redeploy

Data	Survives?	Location
Web app HTML + vouchers	✅	GitHub (in code)
Static POI database	✅	GitHub (in code)
Trip context / itinerary	✅	GitHub (in code)
ToDo checkbox states	✅	Client localStorage
Chat message display	✅	Client localStorage
Chat session ID	✅	Client localStorage
Server chat context	❌	Server memory
Custom POIs	❌	Server memory
WanderGuide user registrations	❌	Server memory
Alert history	❌	Server memory
Broadcast queue	❌	Server memory

Practical impact: Once code changes stop (trip starts), the server runs continuously and all in-memory data persists for the trip duration. Users need to re-register Pushover and re-share Telegram location only if Railway redeploys.

Cost Summary (for 12-day trip)

Service	Cost
GitHub Pages	Free
Railway Hobby	~$5/month
Anthropic Claude API	~$10-20 (estimated usage)
Groq Whisper	Free
Pushover	$5 one-time per device (4 devices = $20)
Telegram	Free
Total	**~$35-45**

Generated April 17, 2026